Partner Enablement Lab Assignment

Version 1.2 — Sep 2026

A practical build assignment for partners delivering with the ElevenAgents Platform

Before you start

You should have received an **enablement pack** containing:

- An index document including enablement resources
- ElevenLabs workspace with activated trial subscription.
- **Five reference agents in your workspace** — workflows, tools, and prompts you can examine in the dashboard. Study these before designing your own.

If those are missing, please reach out to your partner manager.

Prerequisites

- A deployment target for the mock tools backend — Vercel, Supabase, Cloudflare, AWS Lambda, FastAPI + ngrok, or your own.

How to use this document

This document defines the **scenario, constraints, and deliverables**. Every design choice — single agent vs workflow, topology, prompt structure, KB partitioning, tool patterns, test coverage — is yours.

The reference agents and enablement resources in the index document are your primary reference. Use them as you design.

Part 0 — Set up the Enablement Assistant

The Enablement Assistant is a chatbot tuned as a senior solutions engineer for the platform. Use it for design review, prompt critique, debugging, and SDK questions throughout.

1. Create a project in Claude Projects, ChatGPT Custom GPTs, NotebookLM, or Gemini Gems.

2. Download the `enablement-master-prompt` to an md file and paste it as the project's system prompt.
3. Download the `agent-design-principles` and `SIP Glossary and FAQ` files as md and paste it as project context.
4. Verify it works by asking: *"What's the difference between an attached tool and a dispatch tool node?"* Expect an answer grounded in the design principles.

Part 1 — Scenario and scoping

NovaMart is a fictional online retailer (electronics, home goods). The agent handles inbound voice calls for:

- **Order inquiries** — status, items, expected delivery.
- **Returns** — policy questions, initiating a return.
- **Product questions and recommendations.**
- **Changing the shipping address on a pending order** — a privileged action that must not happen before identity is verified.
- **Escalation** to a human when out of scope or requested.

Write a **half-page scoping doc with**:

- The business outcome this agent moves (CSAT, deflection, time-to-resolution, handle time).
- In-scope categories. Out-of-scope categories and how each is handled.
- Three evaluation criteria — concrete observable outcomes, not adjectives.
- Where this sits on the autonomy-vs-control dial, and why.

Part 2 — Build the agent

A few of the requirements reference mock tools (`lookup_order`, `verify_identity`, `update_shipping_address`) that you'll wire up as part of the build. Their signatures are in Tools to build below; runnable mock implementations and test credentials are in Appendix A.

Behavioral requirements

1. **Handles all five intents** from Part 1.
2. **NovaMart content.** Author a small product catalog (8–10 SKUs across two categories with names, prices, key features), a returns policy (30-day window, receipt required, exclusions for opened electronics), and an FAQ (shipping, warranty, payment methods, store hours).
3. **Order lookup** uses `lookup_order`. The agent collects an order ID before calling.

4. **Verification gates privilege.** The agent must not call `update_shipping_address` or share account-specific information until `verify_identity` has returned success in the current conversation. This must be enforced structurally — not by prompt instruction alone.
5. **Up to 3 verification attempts.** After that, escalate.
6. **Routing on tool output is deterministic.** Branches that depend on `verify_identity` success/failure or `lookup_order` found/not-found must read the tool result directly, not be inferred by an LLM evaluating the conversation.
7. **Guardrails.** Does not discuss competitors. Does not invent policy or order details. Does not read full PII back aloud.

Tools to build

Mock implementations and credentials are in Appendix A.

Tool	Type	Signature
`lookup_order`	server (webhook)	`(order_id: string) → order data or not_found`
`verify_identity`	server (webhook)	`(account_number: string, postcode: string) → { success, customer_name? }`
`update_shipping_address`	server (webhook)	`(account_number: string, order_id: string, new_address: string) → confirmation`
`end_call`	system	—
`transfer_to_number`	system (optional)	—

Part 3 — Tests, Evaluation Criteria and Data Collection Items

Build a test suite that would catch regressions before they ship. See `agent-design-principles` for the available test types and patterns.

Your suite should cover:

- The happy path for each intent.
- The verification flow includes a retry and a lockout.
- Cross-flow — intent switch mid-conversation.
- Refusal of guardrail violations (override attempts, competitor questions, off-scope requests).
- Negative assertions — bugs where the failure is the agent doing something it shouldn't (e.g., calling a privileged tool before verification succeeded).

Iterate prompts and workflow until the suite is green.

Create evaluation criteria and data collection items that you think would be useful for NovaMart's agent, based on the described scenario and Behavioral requirements

---

Part 4 — Deploy and roll out

Pick one deployment channel

Whichever is closest to your real customer engagements.

- **Widget** — embed on an HTML page; configure appearance.
- **Phone** — connect via Twilio or a SIP trunk number. See `SIP Glossary and FAQ` for reference.
- **API** — start a conversation programmatically via the SDK.

Branched rollout

Create a second branch of your agent with one meaningful difference (different LLM, tightened guardrail, reworded recovery path, etc.). Set a traffic split. Run enough conversations to see both branches reflected in Conversation Analysis.

Part 5 — Production-readiness review

Open three of your test conversations in Conversations Analysis. Find **one** issue worth fixing before going live. Fix it. Add a regression test that would have caught it. You can also try the Agent Coaching feature for suggested changes.

When you get stuck, use the following resources in that order

1. **The five reference agents** in the enablement pack.
2. **The Enablement Assistant** from Part 0.
3. **The ElevenAgents** official documentation
4. **Support from ElevenLabs FDE.**
5. **Your partner contact** for contractual questions, feature gating, enterprise SLAs, pricing, HIPAA model eligibility.

Part 6 - Submission

Submit through the Partner Enablement Lab submission form. You will need:

1. **Agent ID.**
2. **A short demo walkthrough** with a demo conversation end-to-end plus a walkthrough of your design — workflow, prompts, tool choices, and the *why* behind them. Any format works (Loom, screen recording, unlisted video), as long as it is a shareable link open to anyone with the link.

The form also captures your deployment channel, rollout branch difference, and the production-readiness fix from Part 5, so have those ready. Do not email submissions; anything sent to the inbox will be redirected to the form.

Appendix A — Mock backend reference implementation

The webhook tools each need an HTTPS endpoint returning deterministic JSON. You can host these in a system of choice that'd be suitable for demos.

Common lean options:

- **Supabase Edge Functions** (Deno/TS).
- **Cloudflare Workers** (JS/TS) — fast cold start, free tier.
- **Vercel serverless** (JS/TS/Python) — one-command deploy.
- **AWS Lambda + API Gateway**.
- **FastAPI / Flask + ngrok** for local dev.
- **Any backend you already operate.**

The reference below is JavaScript on Vercel because it's one of the easiest and developer friendly platforms for creating a backend. The handler shapes ports to any of the platforms above.

Do not reuse these endpoints for any real deployment. No auth, no rate limiting, no state. Feel free to come up with your own ones, if desired.

Setup on Vercel

```shell
mkdir novamart-mock-backend && cd novamart-mock-backend
npm init -y
mkdir api
# create the three files below under api/
npx vercel        # first run: login + project setup
npx vercel --prod # subsequent deploys
```

Vercel returns a URL like `https://novamart-mock-backend.vercel.app`. The endpoints are:

- `POST /api/lookup_order`
- `POST /api/verify_identity`
- `POST /api/update_shipping_address`

No auth header needed for this assignment.

For Supabase Edge Functions: `supabase/functions/<name>/index.ts`, deploy via `supabase functions deploy`. For Cloudflare Workers: one Worker per endpoint via `wrangler deploy`. Handler bodies below port directly.

Test credentials

Account number	Postcode	Customer	Use for
`10000001`	`94107`	Jordan Rivera	Happy path
`10000002`	`10001`	Sam Patel	Cross-flow
Anything else	—	—	Retry / lockout (returns `success: false`)

Order ID	Status	Use for
12345	shipped	Happy path
67890	processing	Second order
Anything else	—	Not-found path

api/lookup_order.js

```javascript
const ORDERS = {
  "12345": {
    order_id: "12345",
    status: "shipped",
    items: [{ name: "Nova Headphones X1", qty: 1, price: 149.00 }],
    expected_delivery_date: "2026-05-22",
    shipping_address: "742 Evergreen Terrace, Springfield",
    fulfillment_center: "FC-WEST-03"
  },
  "67890": {
    order_id: "67890",
    status: "processing",
    items: [{ name: "Nova Smart Speaker Mini", qty: 2, price: 79.00 }],
    expected_delivery_date: "2026-05-25",
    shipping_address: "1600 Pennsylvania Ave NW, Washington DC",
    fulfillment_center: "FC-EAST-01"
  }
};

export default function handler(req, res) {
  const { order_id } = req.body || {};
  if (!order_id) return res.status(400).json({ error: "order_id required" });
  const order = ORDERS[order_id];
  if (!order) return res.status(200).json({ status: "not_found", order_id });
  return res.status(200).json(order);
}
```

api/verify_identity.js

```javascript
const IDENTITIES = {
  "10000001": { postcode: "94107", customer_name: "Jordan Rivera" },
  "10000002": { postcode: "10001", customer_name: "Sam Patel" }
};
```

```
export default function handler(req, res) {
  const { account_number, postcode } = req.body || {};
  const record = IDENTITIES[account_number];
  if (!record || record.postcode !== postcode) {
    return res.status(200).json({ success: false });
  }
  return res.status(200).json({
    success: true,
    customer_name: record.customer_name,
    account_number
  });
}
```

Returns `200` with `success: false` on auth failure rather than a 4xx, so the workflow can branch on the semantic result.

api/update_shipping_address.js

JavaScript

```
export default function handler(req, res) {
  const { account_number, order_id, new_address } = req.body || {};
  if (!account_number || !order_id || !new_address) {
    return res.status(400).json({ error: "account_number, order_id, new_address
required" });
  }
  return res.status(200).json({
    success: true,
    order_id,
    new_address,
    updated_at: new Date().toISOString()
  });
}
```

Stateless — every call succeeds with the new address echoed back so the agent can confirm and tests can run repeatedly.